

Node.js

Node.js is a JavaScript runtime environment that allows us to run JavaScript code outside a web browser.

JavaScript runs inside browsers such as Chrome or Firefox. Node.js allows JavaScript to run on a server, so we can use it to build:

Web servers

REST APIs(Representational State Transfer Application Programming Interface)

Web applications

Real-time applications

Command-line tools

Backend services

Node.js is built on Google Chrome's V8 JavaScript engine.

Traditionally:

Browser

↓

JavaScript → DOM → HTML/CSS

With Node.js:

JavaScript

↓

Node.js Runtime

↓

Operating System

↓

Files / Network / Database / Server

Simple example

Create a file called app.js:

```
console.log("Hello, Node.js!");
```

Run it from the terminal:

```
node app.js
```

Output:

```
Hello, Node.js!
```

Here, JavaScript is being executed directly by Node.js instead of by a browser.

Node.js is popular because it is:

Fast

Node.js uses the V8 JavaScript engine, which compiles JavaScript into machine code efficiently.

Asynchronous

Node.js can perform operations such as reading files or communicating with databases without unnecessarily blocking other operations.

Node.js terminal / REPL

When you type:

```
node
```

you enter:

>

This is called the Node.js REPL (Read-Eval-Print Loop).

It's mainly useful for quick experiments:

```
> 10 + 20
```

```
30
```

```
> console.log("Hello")
```

```
Hello
```

```
Undefined
```

The undefined is the return value of console.log() being displayed by the REPL. It is not an error.

For example, you can quickly test:

```
> setTimeout(() => console.log("Done"), 2000)
```

```
Timeout { ... }
```

```
Done
```

You don't need a file for this.

Regular terminal

For an actual program, you normally create:

app.js

with:

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Data received");
```

```
}, 2000);
```

```
console.log("End");
```

Then from the regular terminal:

```
C:\Users\CBIT\Desktop\node-examples>node app.js
```

Output:

Start

End

Data received

Why use the regular terminal?

Because a real program usually contains many lines of code.

For example:

```
my-project/  
|  
├── app.js  
├── server.js  
├── database.js  
└── package.json
```

You can run:

```
node app.js
```

and Node executes the entire file from beginning to end.

With the REPL, you're manually entering commands one at a time.

Node.js REPL

Regular terminal

> prompt	C:\...> prompt
Quick testing	Running programs
Usually no file	Uses .js files
Experiment with JavaScript	Build real applications
node	node app.js

So Node.js itself runs your JavaScript in both cases. The difference is that the REPL is an interactive environment, while the regular terminal is where you tell Node to execute a complete JavaScript file.

For example:

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Data received");  
}, 2000);
```

```
console.log("End");
```

How to run it

Install Node.js if you haven't already.

Save the code as app.js.

Open a terminal in the same folder.

```
C:\Users>cd C:\Users\CBIT\Desktop  
C:\Users\CBIT\Desktop>cd node-examples  
C:\Users\CBIT\Desktop\node-examples>node app.js  
Start  
End  
Data received
```

Run:

node app.js

Output:

Start
End
Data received

The `setTimeout()` operation does not stop the rest of the program from executing.

To execute in node js terminal type the following code in the terminal

```
((() => {  
    console.log("Start");  
  
    setTimeout(() => {  
        console.log("Data received");  
    }, 2000);  
  
    console.log("End");  
}))()
```

Start
End
Data received

For applications, we should create .js files:

Type this in terminal for execution

EXAMPLE : node app.js (filename)

Variables

Filename: variables.js

```
const name = "Rahul";  
const age = 20;  
const marks = 85;
```

```
console.log("Name:", name);  
console.log("Age:", age);  
console.log("Marks:", marks);
```

```
if (marks >= 90) {  
    console.log("Grade: A+");  
} else if (marks >= 80) {  
    console.log("Grade: A");  
} else if (marks >= 70) {  
    console.log("Grade: B");  
} else {  
    console.log("Grade: C");  
}
```

Run:

```
node variables.js
```

Use const when reassignment isn't needed.

Functions

[function.js](#)

```
function add(a, b) {  
  return a + b;  
}
```

```
function subtract(a, b) {  
  return a - b;  
}
```

```
function multiply(a, b) {  
  return a * b;  
}
```

```
console.log("Addition:", add(10, 20));  
console.log("Subtraction:", subtract(20, 10));  
console.log("Multiplication:", multiply(10, 5));
```

Run

```
node function.js
```

Arrow functions

arrow.js

```
const add = (a, b) => a + b;
```

```
const square = number => number * number;
```

```
console.log(add(10, 20));  
console.log(square(5));
```

Arrow functions are particularly common in Node.js callbacks.

Objects

```
const student = {  
  id: 101,  
  name: "Rahul",  
  branch: "CSE",  
  marks: 85  
};
```

```
console.log("ID:", student.id);  
console.log("Name:", student.name);  
console.log("Branch:", student.branch);  
console.log("Marks:", student.marks);  
console.log(student.name);
```

Arrays

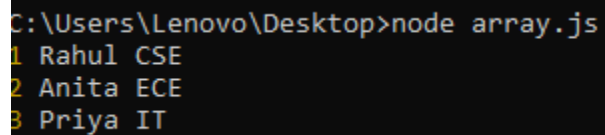
```
const students = [  
  {  
    id: 1,  
    name: "Rahul",  
    branch: "CSE"  
  },  
  {  
    id: 2,  
    name: "Anita",  
    branch: "ECE"  
  },  
  {
```

```

        id: 3,
        name: "Priya",
        branch: "IT"
    }
];

students.forEach(student => {
    console.log(
        student.id,
        student.name,
        student.branch
    );
});

```



```

C:\Users\Lenovo\Desktop>node array.js
1 Rahul CSE
2 Anita ECE
3 Priya IT

```

Destructuring

```

const student = {
    name: "Rahul",
    age: 20
};

const { name, age } = student;

console.log(name);
console.log(age);

```

Synchronous and Asynchronous Programming

Synchronous execution

```
console.log("Start");
```

```
console.log("Processing...");
```

```
console.log("End");
```

Output:

Start

Processing...

End

The statements execute sequentially.

Asynchronous execution

setTimeout()

Consider:

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Data received");  
}, 2000);
```

```
console.log("End");
```

Output:

Start

End

Data received

Start

↓

schedule timer



End



2 seconds



Data received

Because `setTimeout()` schedules a callback to execute later.

Callbacks

A callback is a function passed to another function to be executed later.

Example:

```
function processData(callback) {  
    console.log("Processing data...");  
  
    setTimeout(() => {  
        console.log("Data processing completed");  
        callback();  
    }, 2000);  
}
```

```
function finished() {  
    console.log("Callback executed");  
}
```

```
processData(finished);
```

Output:

Processing data...

Data processing completed

Callback executed

Another example:

```
setTimeout(() => {  
  console.log("Finished");  
}, 2000);
```

The arrow function is the callback.

Promises

Callbacks can become difficult when there are many asynchronous operations.

So we use Promises.

A Promise represents a future result.

A Promise is an object that represents the future result of an asynchronous operation.

For example, when Node.js reads a file, the result isn't available immediately. A Promise says: "When the operation finishes, do this."

Simple example

```
const fs = require("fs").promises;
```

// require("fs") → loads Node.js's built-in File System module

.promises → gives you the version of the file-system functions that work with Promises.

```
//
```

```
fs.readFile("data.txt", "utf8")  
  .then((data) => {  
    console.log(data);  
  })
```

```
.catch((error) => {  
  console.log(error);  
});
```

Here:

- `.then()` → runs when the operation succeeds
- `.catch()` → runs when the operation fails

Using `async/await`

Promises are commonly used with `async/await` because the code is easier to read:

```
const fs = require("fs").promises;
```

```
async function readFile() {  
  try {  
    const data = await fs.readFile("data.txt", "utf8");  
    console.log(data);  
  } catch (error) {  
    console.log(error);  
  }  
}
```

```
readFile();
```

Promise

Start operation



Promise



Success Failure



`.then()` `.catch()`

Async/Await

async/await provides a cleaner way to work with Promises.

Example:

```
function getData() {  
  return new Promise(resolve => {  
    setTimeout(() => {  
      resolve("Data received");  
    }, 2000);  
  });  
}
```

```
async function main() {  
  console.log("Starting...");  
  
  const data = await getData();  
  
  console.log(data);  
  
  console.log("Finished");  
}
```

main();

Output:

Starting...

Data received

Finished

Program Flow

main()



"Starting..." printed
↓
getData() called
↓
Promise created
↓
setTimeout starts
↓
Wait 2 seconds
↓
resolve("Data received")
↓
await finishes
↓
data = "Data received"
↓
"Data received" printed
↓
"Finished" printed

async
↓
function returns Promise

await
↓
wait for Promise result

await doesn't block the Node.js event loop; it suspends that async function until the Promise settles.

Node.js Event Loop

Consider:

```
console.log("1");
```

```
setTimeout(() => {  
  console.log("2");  
}, 0);
```

```
console.log("3");
```

Output:

1

3

2

Why doesn't 2 execute immediately?

Because the timer callback is scheduled for later.

Promise vs Timer

eventloop.js

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Timer");  
}, 0);
```

```
Promise.resolve().then(() => {  
  console.log("Promise");  
});
```

```
console.log("End");
```

Output:

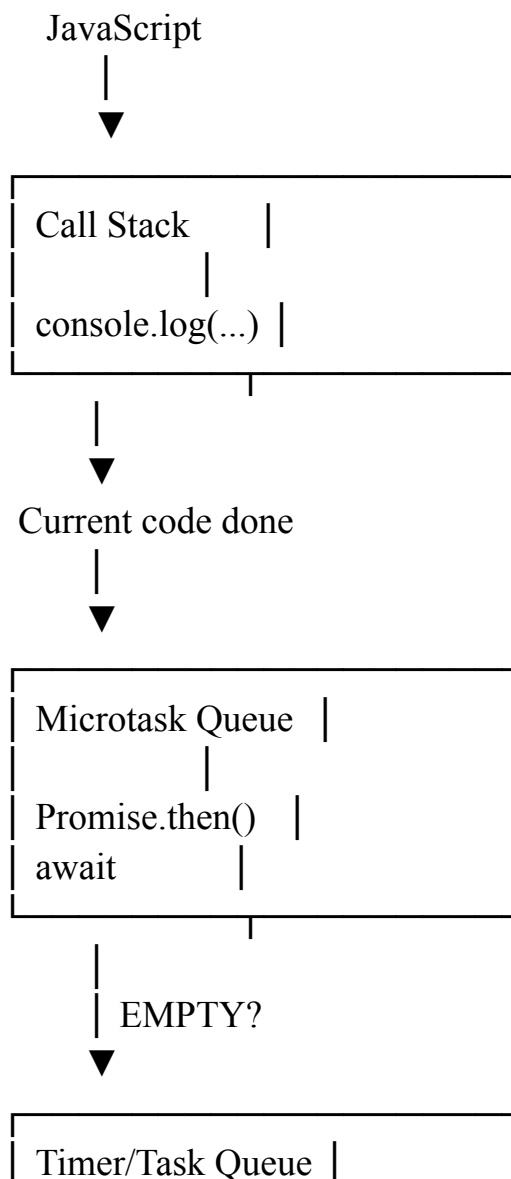
JavaScript has an event loop that decides what code gets executed and when.

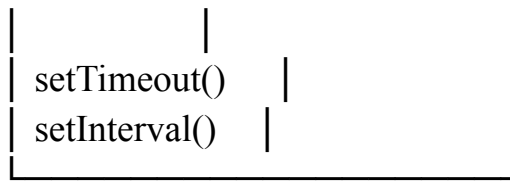
Two important queues are:

- Microtask Queue → Promises, `await`, `.then()`, `.catch()`, `.finally()`
- Timer/Task Queue → `setTimeout()`, `setInterval()`

The important rule is:

After the current synchronous code finishes, JavaScript processes the Microtask Queue before moving to the Timer/Task Queue.





Synchronous code



Microtask Queue



Timer/Task Queue

So Execution order

Start

Timer → waits in the timer queue

Promise is already resolved, so its `.then()` callback is placed in the microtask queue.

Promise → microtask queue

This is normal synchronous code, so it runs before Node.js processes the queued callbacks:

End

At this point:

Microtask queue:

Promise

Timer queue:

Timer

Node.js processes the Promise

Microtasks are processed before moving on to the timer callback:

Promise

Node.js processes the timer

Now the setTimeout callback runs:

Timer

Final order

Start ← synchronous

End ← synchronous

Promise ← microtask

Timer ← timer callback

Synchronous code → Microtasks (Promises) → Timer callbacks

So even though setTimeout has 0 milliseconds, the Promise runs first because Promise callbacks are microtasks, and microtasks are processed before the timer phase.

Start

End

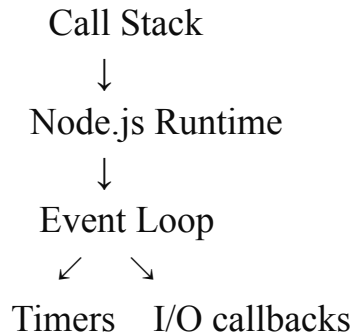
Promise

Timer

A simplified conceptual model:

JavaScript





Node.js can handle many I/O operations without waiting synchronously for each one to finish.

Modules

Modules allow us to divide a large application into smaller files.

Example:

```
project/
├──
├── app.js
└── math.js
```

```
function add(a, b) {
  return a + b;
}
```

```
function subtract(a, b) {
  return a - b;
}
```

```
function multiply(a, b) {
  return a * b;
}
```

```
module.exports = {
```

```
    add,  
    subtract,  
    multiply  
};  
app.js  
const math = require("./math"); // require: Load/import a module or file so I can  
use its code in this file//  
console.log(math.add(10, 20));  
console.log(math.subtract(20, 10));  
console.log(math.multiply(5, 4));
```

Run:

TYPE IN TERMINAL

node app.js

```
app.js  
  ↓  
require("./math")  
  ↓  
Node.js loads math.js  
  ↓  
module.exports  
  ↓  
math object in app.js  
  ↓  
math.add(10, 20)  
  ↓  
30
```

CommonJS and ES Modules

Node.js has two common module systems:

- CommonJS → require() and module.exports
- ES Modules (ESM) → import and export

The example above uses CommonJS, which is commonly seen in Node.js codebases.

ES Modules (ESM) in Node.js

ES Modules are the modern JavaScript module system that uses export and import to share code between files.

Let's use the same example.

1. math.js

```
export function add(a, b) {  
  return a + b;  
}  
  
export function multiply(a, b) {  
  return a * b;  
}
```

Here export makes the functions available to other files.

2. app.js

```
import { add, multiply } from "./math.js";  
  
console.log(add(10, 20));  
console.log(multiply(10, 20));
```

3. Tell Node.js to use ES Modules

In package.json:

```
{  
  "type": "module"  
}
```

Your folder:

```
myapp/
├── app.js
├── math.js
└── package.json
```

4. Execute

```
node app.js
```

Output:

```
30
200
```

CommonJS vs ES Modules

CommonJS	ES Modules
<code>require()</code>	<code>import</code>
<code>module.exports</code>	<code>export</code>
Older Node.js style	Modern JavaScript style

For example:

```
// CommonJS
const math = require("./math");
```

versus:

```
// ES Modules
import { add } from "./math.js";
```

export sends code out of a module, and import brings that code into another module.

Node.js supports both major module styles.

CommonJS

```
const express = require("express");
```

Export:

```
module.exports = something;
```

ES Modules

Import:

```
import express from "express";
```

Export:

```
export default something;
```

Node.js projects uses both

Built-in Node.js Modules

Node.js provides many built-in modules.

fs

path

http
os
events
url
Crypto